

NIRAIVIZHI-Hydrobot Project Review and Roadmap

Executive Summary: NIRAIVIZHI-Hydrobot is an integrated water-quality and health monitoring system combining an autonomous surface-cleaning robot (“HYDROBOT”) with a mobile monitoring application. Its goals are to detect and prevent water pollution, monitor waterborne disease risks in rural communities, and improve public health. The codebase is a Flutter (Dart) app paired with an ESP32-based robot and cloud backend ¹ ². Current development is an initial prototype: source files (e.g. *main.dart*, *pubspec.yaml*) suggest a single Flutter project (100% Dart) with design documents, requirements, and minimal UI code ³ ⁴. Many planned features (multi-role login, multi-language support, real-time sensor feeds, alerts, offline mode, etc.) are documented in requirements but not yet fully implemented ⁵ ⁶. The UI/UX is currently rudimentary (no polished assets), and navigation/state logic appear incomplete. This report analyzes the repo’s structure and gaps, then outlines a phased plan to refactor the UI into a production-grade React Native mobile app. We compare libraries (React Native, Expo, React Navigation, UI kits) with pros/cons, and propose architecture, component breakdown, state management, API integration, styling, offline strategy, testing, CI/CD, and security measures. A suggested schedule and QA/release checklist are also provided.

Project Overview and Goals

The **NIRAIVIZHI-Hydrobot** project is described as *“an integrated smart solution designed to monitor water quality, detect pollution risks, and autonomously clean water surfaces”* ¹. It combines two parts: (1) **HYDROBOT**, an autonomous water-surface cleaning robot equipped with sensors (ESP32, ultrasonic obstacle detector, load cell for waste weight, solar/battery power) ⁷, and (2) **HYDROSYNC**, a real-time monitoring mobile application and cloud backend. The app provides live visualization of sensor data (e.g. pH, turbidity, CO₂, GPS location, battery status) along with user alerts. Key objectives include *real-time monitoring of water and system parameters, autonomous waste collection, sustainable solar-powered operation, and cloud-based data analytics* ⁸. In short, NIRAIVIZHI aims to improve environmental sustainability and public health in underserved regions by combining IoT, robotics, and cloud tech.

Stated Goals (from README): - Provide **real-time water quality monitoring** and alerts.

- Enable **autonomous surface-waste removal** via a solar-powered robot.
- Use a **cloud backend** (MongoDB Atlas) for data storage and analytics.
- Support multiple roles (community members, health workers, village leaders, officials) via a smart app interface.

These objectives are confirmed by the README and requirements documents. For example, the README notes: *“NIRAIVIZHI is an integrated smart solution... improving environmental sustainability and public health”* ¹. The “Project Overview” describes the robot paired with a monitoring app providing *“actionable insights and alerts”* ⁹. A list of “Key Objectives” highlights real-time monitoring, autonomous cleaning, solar operation, and scalability ⁸. In sum, the project seeks to **detect, prevent, and respond to water quality issues and related health risks in rural/tribal areas** by merging the HYDROBOT robotic system with the NIRAIVIZHI mobile platform.

Codebase Structure and Tech Stack

The **code repository** (Jeeveshsankar/NIRAIVIZHI) contains a Flutter mobile app and design documents. The repo has only a handful of files (no subfolders): `.gitignore`, `README.md`, `analysis_options.yaml`, `design.md`, `l10n.yaml` (localization config), `main.dart`, `pubspec.yaml`, and `requirements.md`. This indicates all Flutter code is in `main.dart`. The *README* explicitly lists the tech stack: **Flutter** for the app, **Arduino IDE** for the robot firmware, and **MongoDB Atlas** for the backend database ³. The language breakdown is 100% Dart ⁴, confirming a pure Flutter project.

- **Mobile App (Flutter):** The UI/UX is built in Flutter (Android/iOS). The `pubspec.yaml` (not shown due to rate limits) likely includes dependencies for Firebase/Mongo, charts, etc. We saw `analysis_options.yaml` (Dart lint settings) and a `design.md` placeholder. The main Flutter entrypoint is `main.dart`.
- **Robot Firmware:** The project overview mentions HYDROBOT with an ESP32 microcontroller. While the `requirements.md` suggests an Arduino-based firmware, the repo has no Arduino code (perhaps in a separate repo). The Flutter app presumably communicates via Wi-Fi or Bluetooth to get sensor readings from the robot.
- **Backend:** The *README* lists MongoDB Atlas ³. However, the requirements use terms like “Firebase_Backend” and TLS encryption ¹⁰ ⁶, suggesting a cloud API (REST/WebSocket) or Firebase for data/auth. This inconsistency indicates the backend could be flexible (Mongo vs Firebase). In any case, a cloud service is assumed for data storage and user auth.

Dependencies/Structure: Although code was not directly viewable, given the requirements (like multi-language support via `l10n.yaml`), we infer the app intended extensive functionality (login, multi-role UI, charts, notifications). The lack of subfolders hints the code is monolithic in one file, which will need refactoring into components/modules in any rewrite.

Current UI/UX Assessment

No published app screenshots or design mockups are present in the repo. The available *design.md* and *requirements.md* outline intended UI flows and wireframes, but actual UI code seems minimal. We therefore infer UI/UX based on requirements and project context:

- **Screens & Navigation:** Requirements imply many screens (login, dashboards for different roles, sensor data views, forms for symptoms/lab results, settings, alerts). Without code to view, it seems navigation is not yet implemented. We expect a Flutter app with bottom/tab or stacked navigation for roles, but this is not visible. *Likely gap:* no evidence of a Navigation system or multiple screens in the code; UI likely prototypical or incomplete.
- **Visual Design:** There are no image assets. If any Material widgets are used, they aren't visible. The *README* or design doc doesn't specify style; given Flutter, initial UI may be barebones. *Gap:* No consistency styling or branding, no company/app logo, and no accessibility indicators (colors, contrasts).
- **Responsiveness & Mobile Readiness:** Flutter inherently produces responsive UIs, but the code's organization matters. Without testing, we assume current UI isn't optimized. For example, multi-

language support is required ¹¹, but not yet implemented. Offline readiness (important in rural areas) is not evident. There's no mention of responsive layout beyond Flutter basics.

- **Accessibility:** No references to accessibility in code/docs. This is a gap — production apps should use semantic labels, high-contrast themes, and dynamic font scaling.
- **Overall:** The UI/UX is at best a prototype. It likely lacks polished navigation, error states, input validation, and advanced features. We recommend a complete redesign in React Native to meet mobile standards (see later sections for details).

Functional Features (Current vs. Planned)

The **functionality** can be split between what's in the repo and what the requirements document calls for.

Implemented (In Code)

Based on the limited code files, few features seem present:

- **Basic App Shell:** There must be a `main.dart`, possibly with an `App` class. It might show a home screen or login, but this is unknown due to access issues.
- **Data Models:** No visible model classes, but requirements suggest use of sensor readings, user roles, and database models.
- **Forms/Screens:** Since requirements are extensive, likely only placeholders exist (if anything).

Given the 7 commits in the repo (per [63]), it's very early stage. The *issues* are only documentation (Contributing, Code of Conduct) ¹², not bugs or missing features. This implies most features haven't been attempted or errors are not tracked.

Planned (From Requirements)

The `requirements.md` is very detailed (both functional and non-functional). Key planned features include:

- **User Authentication & Roles:** Login screen with four roles (ASHA worker, Community Member, Village Leader, Health Official) ¹³.
- **Multi-Language Support:** 25+ tribal languages plus English/Hindi ¹¹.
- **Real-Time Sensor Data:** Live display of pH, TDS, temperature, turbidity, CO₂ ⁵.
- **WHO Threshold Alerts:** Push notifications and in-app alerts when readings exceed safe limits ¹⁴.
- **Automated Sample Workflow:** Robot collects water samples above threshold and notifies health workers ¹⁵.
- **Symptom Reporting:** ASHA Workers enter patient symptoms via form (age, gender, symptoms) ¹⁶.
- **Lab Result Submission:** Form to upload lab results linked to a sample (for ML outbreak prediction) ¹⁷.
- **ML Outbreak Prediction:** Analyze combined data to predict outbreaks (details unspecified, but noted as feature) ¹⁸.
- **Dashboards/Analytics:** Health officials see full dashboards of trends and all sources ¹⁹.
- **Village Insights:** Village leaders view local water safety stats (likely a subset of dashboard) ²⁰.

- **Community Alerts:** Timely alerts to community members with push notifications ²¹ .
- **Offline Functionality:** Core features (symptom reports, data sync) must work offline and sync later ²² .
- **Security:** Data encryption (TLS), privacy compliance ²³ .
- **Notification Settings:** Users control alert preferences in settings ²⁴ .
- **Performance:** Fast screen transitions (<1s) and reliability ²⁵ .

These requirements cover the full product vision. **None of these advanced features appear implemented yet.** The README lists *features* more generally (“real-time monitoring, autonomous collection, GPS tracking, alerts, historical analytics” ²⁶), but these are target features, not status.

Gaps and Risks: The gap between the requirements and the actual code is enormous. For example, Requirement 12 (offline mode) and 13 (encryption) are critical for rural use, but the code has no mention. Multi-language and role-based UI (Req.1-2) are non-trivial and likely missing. Security (TLS, auth, data privacy) is unaddressed. Performance (fast UI) and accessibility haven’t been considered.

- *Bug/Performance Risks:* Without caching/offline, the app may fail in low-connectivity areas. Data delays could cause missed alerts. Lack of optimization in rendering sensor streams or maps could slow down the UI.
- *Security Risks:* Storing health data requires encryption-at-rest and in-transit. The app should use secure HTTPS and not cache sensitive data. The requirement envisions TLS 1.3 or higher ¹⁰ , but we see no implementation.
- *Feature Gaps:* No integration code for sensors or backend. Lab/ML features need APIs or background services. Multi-language support will demand substantial localization effort.

In summary, **virtually all planned features are currently gaps.** We must rebuild the UI layer and integrate each component. The following sections outline how to do so in a structured manner.

Recommended React-Based UI Libraries

To refactor the app in React Native, we recommend the following libraries and frameworks. Each is compared by pros/cons and prioritized:

- **React Native (Core):** The base framework for native mobile apps using JavaScript/TypeScript. It has a large ecosystem and allows sharing code between Android and iOS. *Pros:* Strong community, native performance, rich native APIs. *Cons:* Requires learning native aspects (bridges, modules); complex state management may be needed.
- **Expo:** A managed workflow on top of React Native. It provides build tools, Over-The-Air updates, and a suite of cross-platform APIs (camera, sensors, notifications). *Pros:* Simplifies setup and deployment, includes Expo Go for instant preview, can build for web also. *Cons:* Initially limited to Expo SDK-supported modules (though Expo’s bare workflow or EAS now allow custom native code); app size can be larger. Expo is “an open-source platform for making universal native apps that run on Android, iOS, and the web,” enabling writing apps with React/JS ²⁷ . **Expo** would speed early development and prototyping, though for full hardware integration one may eject to bare workflow.
- **React Navigation:** The de facto navigation solution for React Native. It supports stack, tab, drawer, and custom navigators. *Pros:* Easy to set up, cross-platform (iOS/Android/web), extensible (you can

write custom navigators), and supports gestures/animations natively ²⁸. *Cons*: A learning curve for nesting navigators; minor performance overhead. With it we can implement role-based screen flows, nested tabs, and deep linking. The docs note: “*React Navigation provides easy-to-use built-in navigators*” with platform-specific UIs ²⁸.

- **UI Component Libraries**: Ready-to-use components for consistent design. Two top choices:
 - **React Native Paper**: A Material Design library for React Native. *Pros*: Many prebuilt components (buttons, cards, dialogs, data tables), theming support, and active maintenance ²⁹. *Cons*: Material style may not fit all branding; theme customization needed.
 - **NativeBase**: A flexible, “utility-first” UI kit (now part of Gluestack) that works across React and React Native ³⁰. *Pros*: Provides many layout and form components, supports theming and responsiveness (mobile/tablet/desktop). It’s very popular (20k+ stars) and praised as a comprehensive UI library. *Cons*: Larger bundle size; some developers prefer Material or Chakra-like systems. Its site says: “*NativeBase is an accessible, utility-first component library that helps you build consistent UI across Android, iOS and Web.*” ³⁰.

We also consider **React Native Elements** or **UI Kitten** as alternatives: both offer cross-platform components. But Paper and NativeBase have stronger communities. For tailoring styles, **Tailwind RN (NativeWind)** is a modern option: a utility-first styling library bringing Tailwind CSS to React Native. *Pros*: Fast styling using Tailwind classes, small footprint; *Cons*: Not a component library (no buttons, etc., only styles). It requires familiarity with Tailwind.

- **Animation: React Native Reanimated 2/3/4** – for advanced animations that run on the UI thread for smooth performance ³¹. *Pros*: Highly performant (up to 120fps) and can tie into gesture events or sensors ³¹. *Cons*: More complex API; but very useful for maps or data visualization transitions. Reanimated is practically standard for high-performance RN apps now. Its docs boast “smooth animations and interactions up to 120 fps and beyond” ³¹.
- **State Management**: For complex app state (user auth, sensor data, offline cache), options include Redux (with redux-persist), MobX, or Context + React Query. Redux is proven but can be boilerplate-heavy. MobX is reactive and simpler for some, but less common. **React Query** (or **RTK Query**) could manage data fetching/caching from an API (especially if using a REST/GraphQL backend), easing offline capabilities.
- **Other Utilities**:
 - **Formik + Yup** (forms and validation) for symptom/lab entry forms.
 - **Axios or Fetch** for API calls, ideally wrapped with error handling.
 - **@react-native-community/netinfo** to detect connectivity (for offline logic).
 - **@react-native-async-storage/async-storage** or **SQLite** for local caching.

Prioritized Choice: Use **React Native + Expo** as the core platform, with **React Navigation** for routing. For UI, choose either **Paper** or **NativeBase** (or even mix: e.g. Paper for simple widgets, NativeBase for layout). Include **Reanimated** for animations, and possibly **Lottie RN** if we need animated icons. Use **Tailwind RN** for quick styling if the team prefers utility classes.

In summary: we recommend a React Native stack with Expo-managed workflow, React Navigation, and a Material/utility UI library (Paper/NativeBase), plus Reanimated for any required animations. These choices balance rapid development and production-readiness. Official docs confirm their maturity (e.g., React Navigation is “*completely customizable*” and supported by industry sponsors ³²; Paper is “*production-ready*” ²⁹).

Implementation Plan

Phase 1: Project Setup & Architecture (Sprint 1)

- **Tasks:** Initialize a React Native project using Expo (or bare RN if custom native modules are needed). Set up TypeScript, folder structure, linting, and version control. Integrate core libraries: React Navigation, chosen UI kit, state management (Redux/MobX/Context), and API client.
- **Architecture:** Define app layers:
 - *UI Layer:* Screens and components in React Native.
 - *State Layer:* Global store or context for user state, sensor data, offline cache.
 - *Data Layer:* Services for API calls (to cloud backend) and local storage.
 - *Backend Integration:* Outline REST/WebSocket endpoints on Firebase/Mongo. (If the backend is not implemented yet, mock it for now.)
- **Deliverables:** Working app shell with basic navigation and login screen. A demo “Hello World” page.
- **Acceptance:** Must have a functioning Expo app that launches on device/emulator, with a placeholder login (no real auth yet). Navigating between two dummy screens should work.

```
gantt
    dateFormat YYYY-MM-DD
    title Project Timeline (Sample 12-week plan)
    section Sprint 1 - Setup
    Project setup & scaffolding      :done, 2026-03-15, 10d
    UI framework & nav integration    :done, 2026-03-25, 4d
    Design system & theming setup     :done, 2026-03-30, 5d
    section Sprint 2 - Core Features
    Authentication & Roles UI         :2026-04-05, 8d
    Sensor data display screen        :2026-04-15, 6d
    Threshold alerts & notifications  :2026-04-23, 6d
    section Sprint 3 - Advanced Features
    Symptom/Lab forms & storage       :2026-05-01, 8d
    Offline support & caching         :2026-05-10, 6d
    Multi-language support           :2026-05-18, 4d
    section Sprint 4 - Testing & Polish
    End-to-end testing & bug fixes    :2026-05-24, 8d
    Performance tuning & accessibility :2026-06-03, 4d
    Deployment prep (CI/CD)          :2026-06-09, 3d
```

Phase 2: Core Functionality (Sprints 2-3)

- **Authentication:** Implement role-based login. E.g. Firebase Auth or API call for credentials. On login, navigate to a role-specific home screen. Use secure storage (Keychain/EncryptedStorage) for tokens. (Cite RN docs for secure store usage.)

- **Navigation:** Set up a stack navigator with screens (Login → Dashboard/Map → Data Details → Settings, etc.). Use `@react-navigation/native` with separate stacks/tabs for each role. For example, admins get a dashboard and charts, health workers get forms, etc.
- **Sensor Data UI:** Create components to fetch and display real-time readings. Possibly use WebSocket or polling from backend. Use charts (e.g. Victory Native, React Native Charts) for historical data. Ensure labels and units (pH unitless, TDS mg/L, etc.) match requirements ⁵. Include color coding (green/yellow/red) based on WHO thresholds ⁶. Use React Native Paper/NativeBase cards and icons for layout.
- **Alerts/Notifications:** Integrate push notifications (Firebase Cloud Messaging or Expo Notifications). The app should listen for backend alerts and display in-app banners (snackbar or alert dialogs). For example, if pH > WHO limit, show a red banner and send a push to users ⁶. Provide a notifications screen or on-screen indicator.
- **Forms:** Build forms for symptom reporting and lab results using Formik and Yup for validation. Fields: name, age, symptoms (checkbox list), and image upload (lab result photo). On submit, send to backend. Use UI library form inputs. Implement offline queue: if offline, store entries in SQLite/AsyncStorage and sync later (based on NetInfo).

Phase 3: Enhancement and Reliability (Sprints 3-4)

- **Multi-Language:** Implement i18n (e.g. using i18next or React Native Localization) to support >25 tribal languages. Provide language selection on first launch as per requirements ¹¹. Ensure all text strings are translatable.
- **Offline Support:** Critical for rural areas. Cache recent sensor data and allow form submissions offline. On reconnect, sync with server. Show an "offline" mode indicator.
- **Security & Data:** Enforce HTTPS/TLS for all network calls. Do not log sensitive data. Use on-device encryption for stored health info (if any). Ensure user passwords are handled securely (never plaintext). Comply with any legal standards (HIPAA-like) as per requirement.
- **Performance:** Optimize lists (use FlatList with keys), minimize re-renders (React.memo), and lazy-load heavy screens. Use Reanimated for smooth gestures (swiping lists, animations between charts). Ensure screen transitions occur quickly (<1s) to meet Req #15 ²⁵. Use profiling tools (e.g. React Native Profiler) to fix bottlenecks.
- **Accessibility:** Use RN's accessibility props (e.g. `accessible`, `accessibilityLabel`). Ensure text scales and colors contrast. Test with screen readers.

Phase 4: Testing, CI/CD, Release (Sprint 4)

- **Testing Strategy:**
 - *Unit Tests:* Write Jest tests for pure functions and Redux reducers.
 - *Component Tests:* Use React Native Testing Library for key screens (e.g. login, sensor display) to verify UI behavior.
 - *Integration:* Test data flows (mock API or local test server).
 - *E2E Tests:* Use Detox or Cypress (via Expo) for end-to-end on device (e.g. login → view data → receive alert).
- **CI/CD:** Set up GitHub Actions or Bitrise/EAS for automated builds. On each push, run lint, tests, and build APK/IPA. Use Expo EAS for cloud builds and Over-The-Air updates. Ensure environment config (API keys, certificates) are secured.
- **Deployment:** Prepare app store metadata, generate signing keys, and privacy policy. The release checklist (below) will ensure all steps are done.

Overall Architecture Diagram:

```

flowchart LR
    subgraph Mobile_App [Mobile App (React Native)]
        direction TB
        A[Login Screen] --> B{User Role?}
        B -->|Community Member| C[Community Dashboard]
        B -->|ASHA Worker| D[Health Worker Dashboard]
        B -->|Village Leader| E[Village Leader View]
        B -->|Health Official| F[Health Official Dashboard]
        C --> G[Sensor Detail]
        D --> H[Symptom Form]
        D --> I[Lab Results Form]
        E --> G
        F --> J[Analytics Charts]
        J --> G
    end
    subgraph Device_IoT [Device & IoT]
        K[HYDROBOT (ESP32)]
        L[Sensors: pH, Temp, TDS, Turbidity, CO2]
        K --> L
        L --> M[WiFi / Cellular Transmitter]
    end
    subgraph Backend_Cloud [Backend & Cloud]
        M --> N[(MongoDB Atlas)]
        M --> O[(Firebase Auth & Messaging)]
        O --> A
        N --> A
    end

```

The mobile app has screens branching by user role. HYDROBOT sends data via a network to the cloud (MongoDB/Firebase). The app pulls from the cloud. Push notifications use Firebase.

Effort Estimation and Milestones

A tentative **timeline** (sprints of ~2 weeks) is outlined above. Below is a summary table of sprints, deliverables, and acceptance criteria:

Sprint	Duration	Deliverables	Acceptance Criteria
1. Setup & Auth	2 weeks	- React Native (Expo) project scaffold - Login/Signup screens - Role-based navigation structure	- App builds and runs on device - User can “log in” (mock auth) and is routed to a role-specific placeholder screen - All libraries installed and lint/tests passing

Sprint	Duration	Deliverables	Acceptance Criteria
2. Sensor Data & Alerts	2–3 weeks	- Sensor data screen (list or dashboard) showing readings - Push notification setup (Firebase Expo) - Alert UI (snackbar/dialog) when data is out of range	- Live data flow simulated/mock is displayed within 5s ⁵ - When a value exceeds threshold, an in-app alert and push notification are shown within 10s ⁶ - Users see color-coded status (green/yellow/red) matching safety levels
3. Forms & Offline	2–3 weeks	- Symptom report form and lab result form - Data persisted offline (AsyncStorage/SQLite) - Sync logic on reconnection - Multi-language UI (at least English & one tribal language)	- ASHA Worker can fill and submit forms; entries saved offline if no connectivity - On restore, queued entries auto-sync - UI text changes based on language selection ¹¹ - All form fields validate and error messages shown for missing inputs
4. Polish, Testing & Deploy	2 weeks	- Complete QA testing (unit, integration, E2E) - Accessibility fixes - Performance optimizations - CI/CD pipeline (GitHub Actions/EAS) - App store builds and docs	- >90% unit test coverage; no critical bugs open - App meets performance criteria (transitions <1s) ³³ - Accessibility compliance (WCAG AA colors, screen reader labels) - CI pipeline automatically builds APK/IPA on main branch - Ready for Beta/Production release

Estimated effort: Assuming a small team (2–3 developers), this plan would take roughly 2–3 months of work (including reviews and buffer). With experienced React Native devs, the above sprints are feasible. Each deliverable is tied to acceptance tests (e.g. “login works,” “data displays correctly,” etc.) to ensure quality.

Risk Assessment and Mitigation

Risk	Impact	Mitigation
Hardware/Integration Delays: Dependence on HYDROBOT data feed (sensors) may delay app testing.	High	Mock sensor API endpoints during development. Use a simulator or stub service to generate data. Coordinate with hardware team to get interface specs early.
Connectivity Issues: Rural areas have intermittent Internet.	High	Implement offline caching and sync (Redux Persist/SQLite). Use efficient data updates (only fetch changes). Minimize data usage. Provide clear offline UX hints. Test under poor network conditions.

Risk	Impact	Mitigation
Data Privacy/Security: Handling health data requires compliance.	High	Enforce TLS for all network calls (React Native Fetch is HTTPS). Use secure storage for credentials. Anonymize or encrypt sensitive data. Consult legal guidelines (e.g. GDPR, HIPAA). Regularly update dependencies.
Performance on Older Devices: Raspberry Pi/ESP32 limits or low-end phones can struggle.	Medium	Profile and optimize code. Use lightweight components. Enable GPU rendering (Hermes engine). Offload heavy work (e.g. ML) to cloud. Limit background tasks.
Scope Creep: Extensive features (ML outbreak prediction, dashboards) can overrun schedule.	Medium	Prioritize core features first (monitoring & alerts). Defer advanced analytics/ML to post-release. Use Agile sprints and backlog grooming to manage scope.
Team Skill Gaps: New libraries (Reanimated, offline DB) may have learning curve.	Medium	Allocate time for spike/prototyping. Use well-documented libraries. Engage experts or trainings if needed.

Mitigations include early prototyping, heavy use of testing (to catch UI bugs), and staging deployments. For example, to mitigate security risk, we will require all data transfers use HTTPS/TLS (React Native's networking supports this by default). The acceptance criteria for security (e.g. TLS 1.3 ²³) will be verified by code review and pen-testing.

Developer Tools, Libraries, and Samples

Recommended Tools: - **VS Code** (with React Native extensions), - **Expo CLI** (for managed builds), - **Android Studio & Xcode** (for emulators), - **Flipper** (React Native debugging), - **GitHub/GitLab** (version control and CI), - **Postman/Insomnia** (API testing).

Libraries to Use: - **React Navigation** (routing) ²⁸, - **React Native Paper** or **NativeBase** (UI kit) ²⁹ ³⁰, - **Reanimated** (animations) ³¹, - **Redux** or **MobX** (state), - **Axios** (network calls), - **Formik + Yup** (form handling), - **i18next** (localization), - **Redux Persist / SQLite** (offline cache), - **React Native Testing Library + Jest** (testing).

Code Snippet (Example): A basic React Navigation stack setup for login and role-based flow:

```
// App.tsx (simplified)
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import LoginScreen from './src/screens/LoginScreen';
import DashboardScreen from './src/screens/DashboardScreen';
import { UserContextProvider, useUser } from './src/context/UserContext';
```

```

const Stack = createNativeStackNavigator();

function AppNavigator() {
  const { role } = useUser(); // e.g. 'community', 'asha', etc.

  return (
    <Stack.Navigator initialRouteName="Login">
      <Stack.Screen name="Login" component={LoginScreen} />
      { /* After login, navigate to role-specific initial screen */ }
      <Stack.Screen
        name="Dashboard"
        component={DashboardScreen}
        options={{ headerShown: false }}
      />
      { /* Additional screens can be defined here */ }
    </Stack.Navigator>
  );
}

export default function App() {
  return (
    <UserContextProvider>
      <NavigationContainer>
        <AppNavigator />
      </NavigationContainer>
    </UserContextProvider>
  );
}

```

This illustrates using React Navigation's Stack Navigator. After a successful login (handled in `LoginScreen`), we'd navigate to a "Dashboard" whose content depends on `role`. (In production code, we would protect routes and show different tabs for each role.)

QA and Release Checklist

QA Checklist: - [] **Functionality:** All features (login, data display, forms, alerts) work per requirements. - [] **Navigation/UX:** Flow matches user expectations, no dead-ends. - [] **Error Handling:** Appropriate feedback for failures (network errors, input validation errors). - [] **Performance:** App launches <3s; screen transitions <1s³³; no UI jank. - [] **Offline:** Core features are usable without network; data syncs correctly. - [] **Localization:** Strings present for all supported languages; switching languages updates UI. - [] **Security:** All API calls HTTPS; no sensitive logs; permissions (camera, storage) requested properly. - [] **Accessibility:** Text readable at larger font; contrast meets standards; screen reader labels on buttons/inputs. - [] **Usability:** On-device fonts/icons scaled; layouts adapt to device sizes (phones/tablets).

Release Checklist: - [] **Versioning:** Semantic version bump. - [] **Environment Config:** API endpoints and keys set for production (not dev/test). - [] **App Icons/Splash:** Final artwork added. - [] **Privacy Policy:** Link in app & store metadata. - [] **Store Submission:** Android bundle (AAB) and iOS build archived; developer

certificates set up. - [] **Beta Testing:** Deploy to testers (TestFlight/Play Internal) and gather feedback. - [] **Metrics/Logging:** Integrate (optional) analytics or crash reporting (e.g. Sentry) for production monitoring. - [] **Documentation:** Update README, usage guides, and code comments.

Each item must be ticked off before final deployment.

Sources: This analysis is based on the project's own documentation and code (README ¹ ², requirements ⁵ ⁶), and official docs of recommended libraries (React Navigation ²⁸, React Native Paper ²⁹, NativeBase ³⁰, Reanimated ³¹, Expo ²⁷). The plan above synthesizes these into a practical, phased approach to building a robust mobile app. The use of community-vetted libraries and extensive testing/CI strategies aims to deliver a polished, production-grade app aligned with the project's goals.

¹ ² ³ ⁴ ⁷ ⁸ ⁹ ²⁶ GitHub - Jeeveshsankar/NIRAIVIZHI: Smart community health monitoring and early warning system for preventing water-borne diseases in rural Northeast India.

<https://github.com/Jeeveshsankar/NIRAIVIZHI>

⁵ ⁶ ¹⁰ ¹¹ ¹³ ¹⁴ ¹⁵ ¹⁶ ¹⁷ ¹⁸ ¹⁹ ²⁰ ²¹ ²² ²³ ²⁴ ²⁵ ³³ NIRAIVIZHI/requirements.md at main · Jeeveshsankar/NIRAIVIZHI · GitHub

<https://github.com/Jeeveshsankar/NIRAIVIZHI/blob/main/requirements.md>

¹² GitHub · Where software is built

<https://github.com/Jeeveshsankar/NIRAIVIZHI/issues>

²⁷ GitHub - expo/expo: An open-source framework for making universal native apps with React. Expo runs on Android, iOS, and the web. · GitHub

<https://github.com/expo/expo>

²⁸ ³² React Navigation

<https://reactnavigation.org/>

²⁹ React Native Paper

<http://oss.callstack.com/react-native-paper/>

³⁰ NativeBase: Universal Components for React & React Native

<https://nativebase.io/?ref=producthunt>

³¹ React Native Reanimated

<https://docs.swmansion.com/react-native-reanimated/>